

# Resumen de Programación Avanzada

## 1 CONTENIDO

---

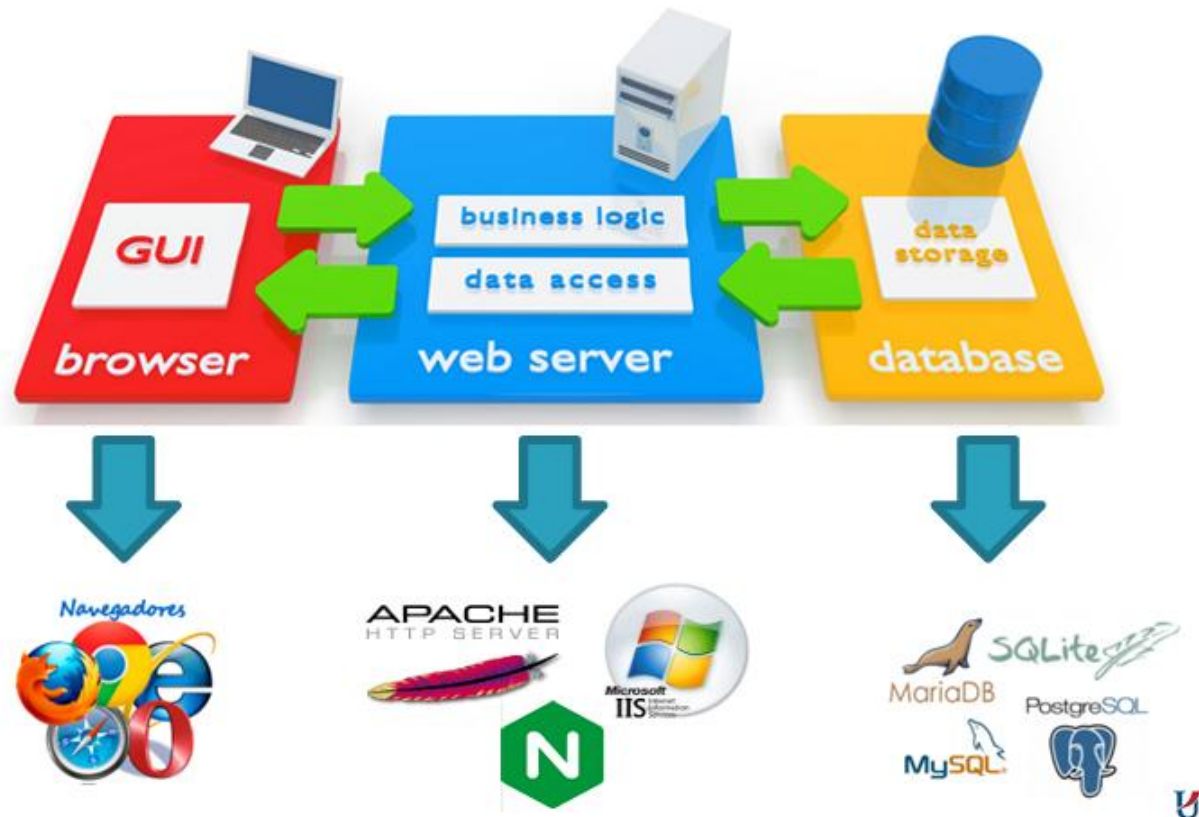
|       |                                                           |    |
|-------|-----------------------------------------------------------|----|
| 1     | Clase 1: Funcionamiento de la Web .....                   | 4  |
| 1.1   | Navegadores Web .....                                     | 4  |
| 1.2   | Servidor Web .....                                        | 4  |
| 1.3   | Base de datos .....                                       | 5  |
| 1.4   | Front-end / Back-end.....                                 | 6  |
| 1.5   | XAMPP.....                                                | 6  |
| 1.6   | Funcionamiento web.....                                   | 6  |
| 1.7   | Localizador Uniforme de Recursos (URL).....               | 7  |
| 1.8   | DNS (Domain Name System) .....                            | 7  |
| 2     | Clase 2: .....                                            | 7  |
| 2.1   | HTML y CSS.....                                           | 7  |
| 2.2   | Formas de incluir CSS.....                                | 7  |
| 2.3   | Evolución HTML - W3C.....                                 | 8  |
| 2.4   | Etapas de Madurez en la vía de recomendación .....        | 9  |
| 2.5   | CSS: Estándar modular .....                               | 9  |
| 2.6   | W3C y Whatwg.....                                         | 9  |
| 2.7   | WHATWG.....                                               | 10 |
| 2.8   | Conclusión.....                                           | 10 |
| 3     | Clase 3: Navegadores.....                                 | 10 |
| 3.1   | JS Historia .....                                         | 10 |
| 3.2   | Timeline .....                                            | 11 |
| 3.3   | ECMA .....                                                | 11 |
| 3.4   | JS en el Cliente .....                                    | 11 |
| 3.5   | JS en Escritorio.....                                     | 12 |
| 3.6   | IETF Internet Engineering Task Force.....                 | 12 |
| 4     | Clase 4: IDEs .....                                       | 13 |
| 4.1   | Definición e Importancia del IDE .....                    | 13 |
| 4.2   | Características y Componentes Principales de un IDE ..... | 14 |
| 4.3   | Tipos de IDE .....                                        | 14 |
| 4.3.1 | IDE locales (o de escritorio):.....                       | 14 |
| 4.3.2 | IDE en la nube (o Web):.....                              | 14 |
| 4.4   | Estadísticas de Uso de IDE .....                          | 14 |
| 5     | Clase 5: UX y UI.....                                     | 15 |
| 5.1   | Definición de UX y UI.....                                | 15 |

|       |                                                            |    |
|-------|------------------------------------------------------------|----|
| 5.2   | Disciplinas Relacionadas y Enfoque.....                    | 15 |
| 5.2.1 | Design Thinking: Un proceso iterativo de 5 fases: .....    | 15 |
| 5.3   | Diseño de Interfaz (UI) .....                              | 16 |
| 5.4   | Elementos y Principios del Diseño Visual .....             | 16 |
| 5.4.1 | Elementos del Diseño Visual (7):.....                      | 16 |
| 5.4.2 | Principios del Diseño (6): .....                           | 16 |
| 5.5   | Prototipo.....                                             | 17 |
| 6     | Clase 6: Control de versiones.....                         | 17 |
| 6.1   | Conceptos Fundamentales del Control de Versiones .....     | 17 |
| 6.1.1 | Puntos Clave de un VCS .....                               | 17 |
| 6.2   | Tipos de Sistemas de Control de Versiones.....             | 18 |
| 6.2.1 | Sistemas de Control de Versiones Centralizados (CVCS)..... | 18 |
| 6.2.2 | Sistemas de Control de Versiones Distribuidos (DVCS) ..... | 18 |
| 6.3   | Git y Plataformas Colaborativas .....                      | 18 |
| 6.3.1 | Historia: .....                                            | 18 |
| 6.4   | Flujo Básico y Clientes de Git .....                       | 19 |
| 6.4.1 | Flujo de Trabajo (Áreas de Git) .....                      | 19 |
| 6.4.2 | Clientes GUI e Integración con IDE.....                    | 19 |
| 7     | Clase 7: Seguridad en APPs web.....                        | 20 |
| 7.1   | Fundamentos de la Seguridad (Triángulo CIA) .....          | 20 |
| 7.2   | Riesgos y Punto de Equilibrio.....                         | 20 |
| 7.3   | Aspectos de Seguridad en una Aplicación Web .....          | 21 |
| 7.4   | Cifrado vs. Hash .....                                     | 22 |
| 7.5   | OWASP Top 10.....                                          | 22 |
| 7.6   | Recomendaciones de Alto Nivel .....                        | 22 |
| 8     | Clase 8: Laravel .....                                     | 22 |
| 8.1   | Qué es .....                                               | 22 |
| 8.2   | Características .....                                      | 22 |
| 8.3   | Estructura.....                                            | 23 |
| 8.3.1 | MVC: .....                                                 | 23 |
| 8.4   | Funcionamiento.....                                        | 23 |
| 8.5   | Laravel como Framework.....                                | 24 |
| 8.5.1 | Instalación .....                                          | 24 |
| 8.5.2 | Para crear un proyecto con laravel es:.....                | 24 |
| 8.5.3 | .env.....                                                  | 24 |
| 8.6   | Artisan - Comandos.....                                    | 24 |
| 8.6.1 | Comandos de servidor y mantenimiento.....                  | 24 |
| 8.6.2 | Comandos de Generación (Scaffolding).....                  | 24 |
| 8.7   | Vistas y Blade .....                                       | 25 |

|        |                                                             |    |
|--------|-------------------------------------------------------------|----|
| 8.7.1  | Subvistas .....                                             | 25 |
| 8.7.2  | Layout Blade.....                                           | 26 |
| 8      | Clase 9: Selenium.....                                      | 26 |
| 8.1    | Testing Automatizado .....                                  | 26 |
| 8.2    | Selenium.....                                               | 26 |
| 8.3    | Componentes de Selenium: .....                              | 26 |
| 8.4    | Selenium Grid y Docker (Arquitectura Distribuida).....      | 26 |
| 8.5    | Selenium en el Contexto de Ciberseguridad y Monitoreo ..... | 26 |
| 9      | Clase 10: Laravel 2: La resurrección.....                   | 26 |
| 9.1    | Configuración de la Base de Datos.....                      | 26 |
| 9.2    | Mapeo Objeto-Relacional (ORM) - Eloquent.....               | 26 |
| 9.3    | Migraciones de Base de Datos .....                          | 27 |
| 9.4    | Inicialización de la Base de Datos (Database Seeding).....  | 27 |
| 9.5    | Constructor de Consultas (Query Builder) .....              | 27 |
| 9.6    | Relaciones entre Modelos con Eloquent ORM.....              | 28 |
| 10     | Clase 11: Seguridad Web Parte 2: Ahora es personal .....    | 28 |
| 10.1   | Seguridad en la Comunicación y Cifrado de Información.....  | 28 |
| 10.2   | Arquitectura de Aplicaciones Web (General) .....            | 28 |
| 10.3   | HTTP vs. HTTPS.....                                         | 28 |
| 10.4   | Evolución de SSL/TLS .....                                  | 29 |
| 10.4.1 | Certificados Digitales (SSL/TLS) .....                      | 29 |
| 10.5   | HTTPS Handshake .....                                       | 30 |
| 10.6   | Let's Encrypt.....                                          | 30 |
| 10.7   | Conclusiones y Tendencias .....                             | 31 |

# 1 CLASE 1: FUNCIONAMIENTO DE LA WEB

---



## 1.1 NAVEGADORES WEB

Son programas / aplicaciones que interpretan la información de distintos tipos de archivos (html, css, js) para visualizar el contenido en el dispositivo que lo ejecuta.

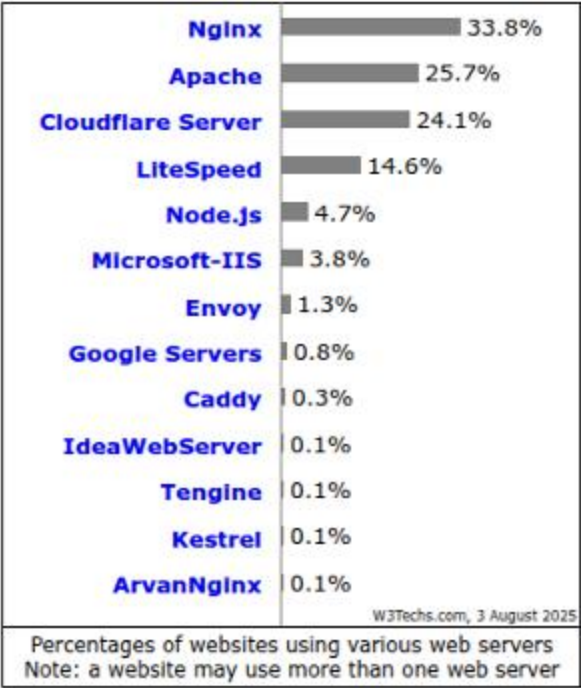
Chrome es el navegador top 1 utilizado en el mundo, seguido de Safari.

## 1.2 SERVIDOR WEB

Un servidor web es un sistema informático que combina hardware y software con el propósito de procesar solicitudes realizadas a través del protocolo HTTP o HTTPS, respondiendo con contenidos digitales como páginas HTML, archivos multimedia o servicios web.

El software interpreta las solicitudes entrantes, accede al contenido solicitado (estático o dinámico), lo encapsula en una respuesta HTTP, y lo envía de vuelta al cliente.

NGINX es el webserver más utilizado.

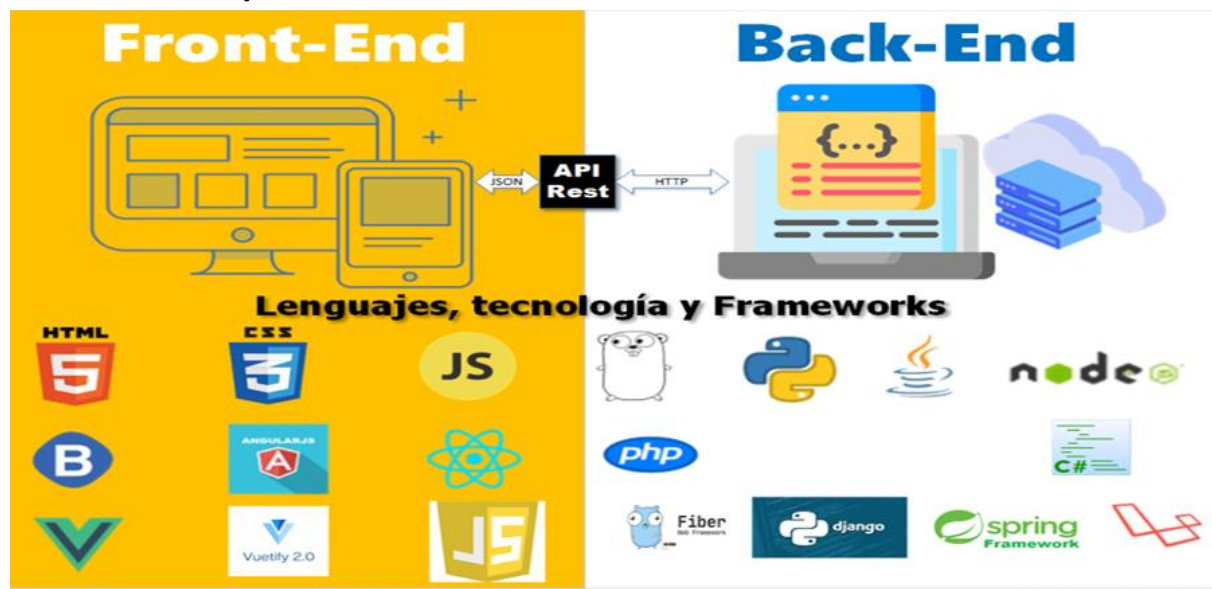


1.3 BASE DE DATOS

Para que te la voy a definir si la viste en BD capo.

| Rank     |          |          | DBMS                 | Database Model             | Score    |          |          |
|----------|----------|----------|----------------------|----------------------------|----------|----------|----------|
| Aug 2025 | Jul 2025 | Aug 2024 |                      |                            | Aug 2025 | Jul 2025 | Aug 2024 |
| 1.       | 1.       | 1.       | Oracle               | Relational, Multi-model ⓘ  | 1220.70  | +3.64    | -37.78   |
| 2.       | 2.       | 2.       | MySQL                | Relational, Multi-model ⓘ  | 915.46   | -25.26   | -111.40  |
| 3.       | 3.       | 3.       | Microsoft SQL Server | Relational, Multi-model ⓘ  | 754.15   | -16.99   | -61.02   |
| 4.       | 4.       | 4.       | PostgreSQL           | Relational, Multi-model ⓘ  | 671.25   | -9.63    | +33.87   |
| 5.       | 5.       | 5.       | MongoDB ⬆            | Document, Multi-model ⓘ    | 395.58   | -8.25    | -25.40   |
| 6.       | 6.       | ⬆7.      | Snowflake            | Relational                 | 178.90   | +2.73    | +42.93   |
| 7.       | 7.       | ⬇6.      | Redis                | Key-value, Multi-model ⓘ   | 147.19   | -2.53    | -5.52    |
| 8.       | 8.       | ⬆9.      | IBM Db2              | Relational, Multi-model ⓘ  | 127.31   | -0.20    | +4.30    |
| 9.       | ⬆12.     | ⬆15.     | Databricks           | Multi-model ⓘ              | 115.82   | +7.78    | +31.36   |
| 10.      | ⬇9.      | ⬇8.      | Elasticsearch        | Multi-model ⓘ              | 114.27   | -4.56    | -15.56   |
| 11.      | ⬇10.     | ⬇10.     | SQLite               | Relational                 | 112.59   | -2.84    | +7.80    |
| 12.      | ⬇11.     | ⬇11.     | Apache Cassandra     | Wide column, Multi-model ⓘ | 108.51   | -0.24    | +11.51   |
| 13.      | 13.      | ⬆14.     | MariaDB ⬆            | Relational, Multi-model ⓘ  | 93.59    | -1.85    | +7.06    |
| 14.      | 14.      | ⬇12.     | Microsoft Access     | Relational                 | 87.76    | -2.71    | -8.61    |

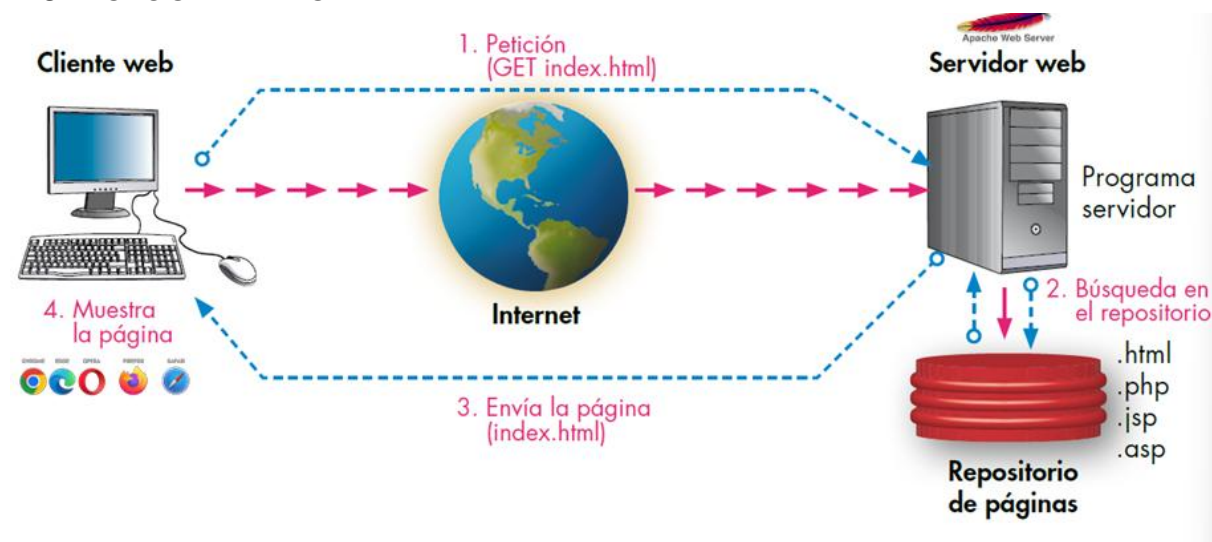
## 1.4 FRONT-END / BACK-END



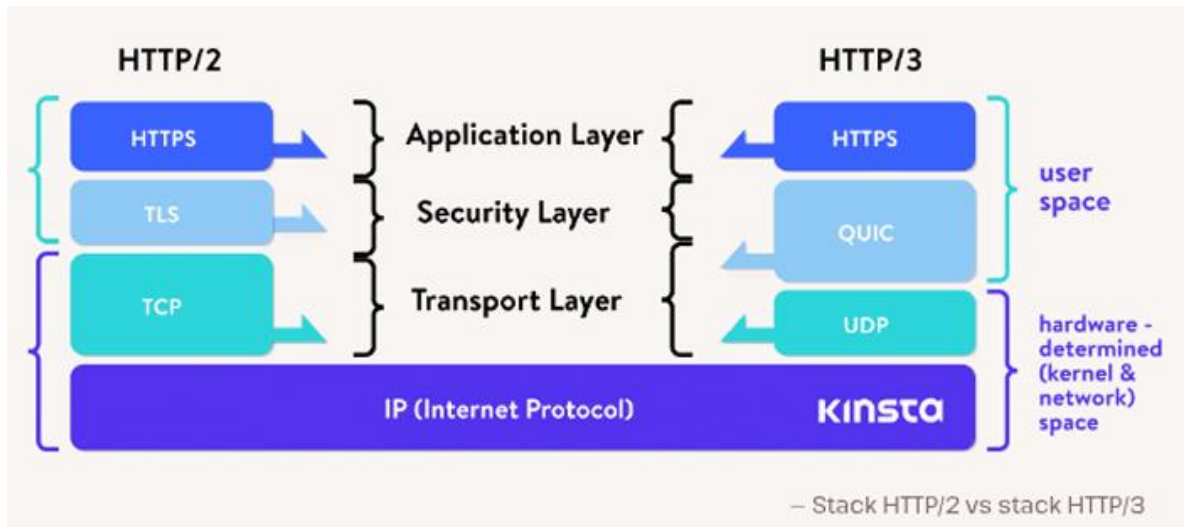
## 1.5 XAMPP

Es un entorno de trabajo, que nos trae las utilidades de PHP, MariaDB, PHPmyAdmin, Pearl, Apache Web Server, y funciona en cualquier sistema operativo.

## 1.6 FUNCIONAMIENTO WEB



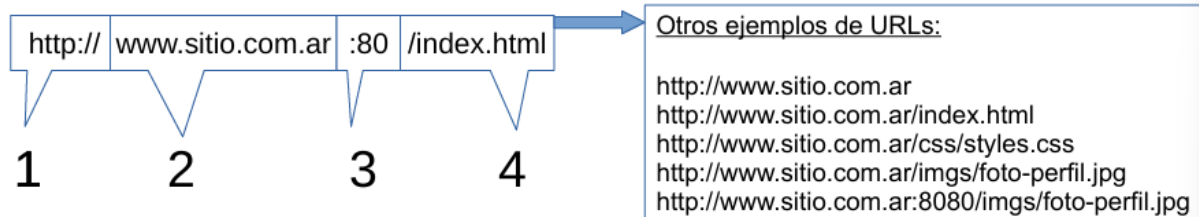
- **HTTP/1.1:** Conexiones individuales por recurso, encabezados duplicados, limitación de concurrencia.
- **HTTP/2:** Multiplexación sobre una única conexión TCP, compresión de encabezados (HPACK), *server push*, priorización de streams.
- **HTTP/3:** Basado en QUIC (sobre UDP), eliminación de bloqueo de cabeza de línea a nivel de conexión, Handshake más rápido, mejor manejo de cambios de red.



HTTP/3 CHECK chequea si un sitio tiene http3. Podemos usar devtools también del navegador propio.

## 1.7 LOCALIZADOR UNIFORME DE RECURSOS (URL)

Al utilizar la URL, estamos solicitando por medio del navegador un recurso alojado en un servidor. Una URL es una secuencia de caracteres estandarizados que permiten identificar unívocamente un recurso perteneciente a una red. Una URL con un esquema HTTP tiene 4 elementos básicos:



El protocolo, dominio, puerto, y el recurso buscado.

## 1.8 DNS (DOMAIN NAME SYSTEM)

Es el servidor que contiene las direcciones IP de los sitios web a los que queremos acceder.

# 2 CLASE 2:

## 2.1 HTML Y CSS

HTML = Lenguaje de Marcado de Hipertexto (HTML) es el código que se utiliza para estructurar y desplegar una página web y sus contenidos. HTML no es un lenguaje de programación, sino de marcado.

CSS = Cascading Style Sheets / Lenguaje de estilos.. Sirve para presentar documentos HTML o XML.

## 2.2 FORMAS DE INCLUIR CSS

1. Con una hoja externa:

```
<head>
  <link rel="stylesheet" href="s
</head>
```

2. Con una hoja de estilos interna:

```
<head>
  <style>
    body {
      background-color: 1
    }
    h1 {
      color: navy;
    }
  </style>
</head>
```

3. Estilos de linea:

```
<body>
  <h1 style="color: blue; text-align: cent
  <p style="font-size: 18px;">Este párrafo
</body>
```

## 2.3 EVOLUCIÓN HTML - W3C

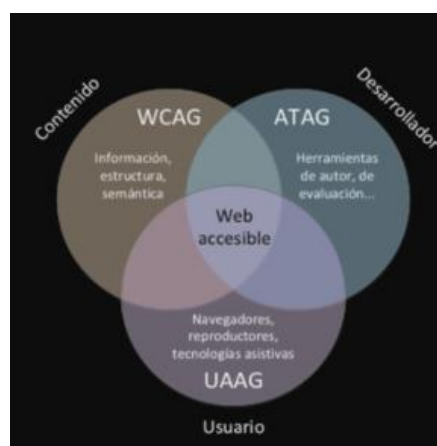
1989: Se inventa la World Wide Web WWW.

1991: Tim Berners-Lee inventa HTML.

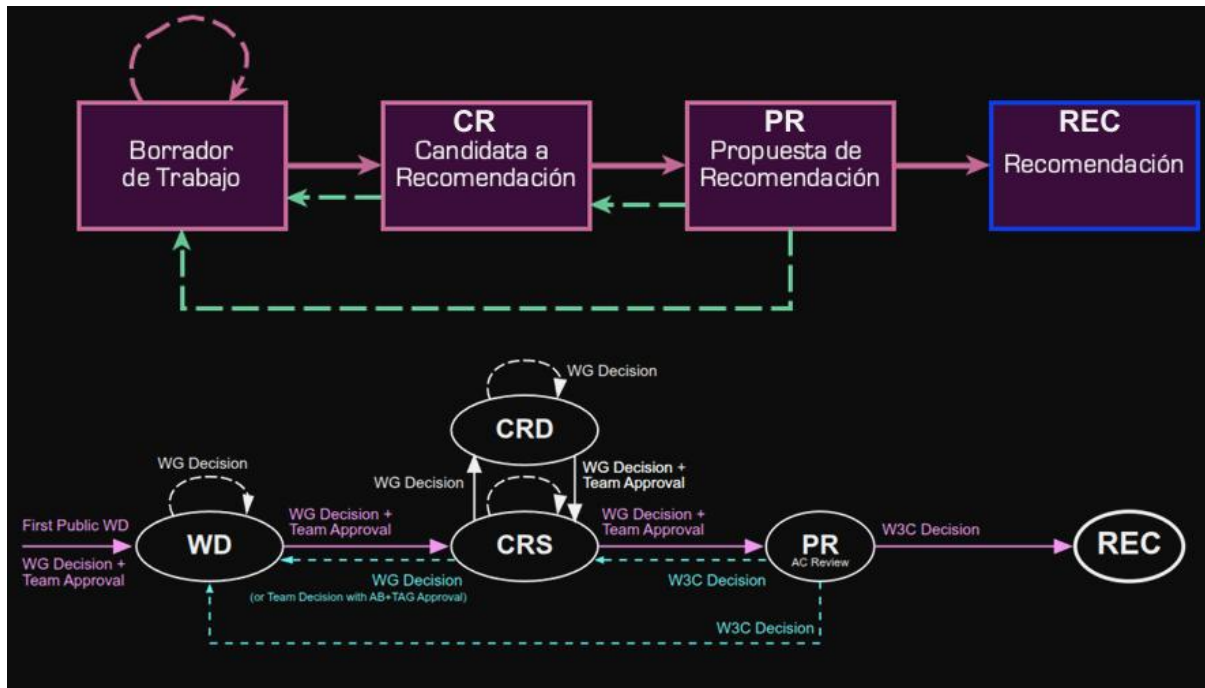
W3C es una organización internacional fundada en 1994 por Tim Berners-Lee. El objetivo era desarrollar estándares como HTML, CSS y XML.

Tienen la Iniciativa de Accesibilidad Web (WAI), que aporta:

1. Web Content Accessibility Guidelines (WCAG): Hacen que los contenidos de la web sean perceptibles, operables, comprensibles y robustos para personas con discapacidad.
2. Authoring Tool Accessibility Guidelines (ATAG): Es para desarrolladores de autoría web.
3. User Agent Accessibility Guidelines (UAAG): Para desarrolladores de agentes de usuario web (navegadores por ejemplo).







## 2.4 ETAPAS DE MADUREZ EN LA VÍA DE RECOMENDACIÓN

- Borradores de Trabajo (Working Drafts): Son documentos expuestos a la comunidad para que expertos externos opinen. No implican consenso total.
- Candidata a Recomendación (Candidate Recommendation): Cuando las especificaciones han alcanzado suficiente madurez y cumplen los requisitos técnicos definidos por el grupo.
  - CRS (Candidate Recommendation Snapshot): versión estable y “congelada”, lista para recolección de feedback y activación del proceso de patentes.
  - CRD (Candidate Recommendation Draft): versión editable, que refleja cambios desde la última CRS pero sin activar proceso de patentes.
- Propuesta de Recomendación (Proposed Recommendation): El Comité Asesor, formado por todos los organismos miembro del W3C, revisa el contenido técnico y los aspectos legales.
- Recomendación (W3C Recommendation): Cuando se consigue el consenso del Comité Asesor, el Director del W3C declara la especificación como estándar oficial.

## 2.5 CSS: ESTÁNDAR MODULAR

Está compuesto de módulos independientes. Cada módulo tiene su ciclo de vida en el proceso W3C. Algunos pueden estar en recomendación y otros en Snapshot.

Esto permite avanzar más rápido, actualizar un módulo puntual sin reescribir el estándar, e implementar parciales en navegadores.

## 2.6 W3C Y WHATWG.

1990: HTML paso por revisiones y experimentaciones alojadas en el CERL y luego en IETF.

1997: Sale HTML 3.2 y HTML 4 el mismo año.

1998: W3C deja de desarrollar HTML, para trabajar en XHTML. También trabajaron en XHTML2 que era incompatible con XHTML y HTML. Se detuvo la evolución de HTML.

2000: Salen tecnologías que buscan dejar atrás a HTML.

2003: Una publicación de XForms hace que se quiera seguir innovando en HTML. Por la buena compatibilidad hacia atrás.

2004: Reabre la evolución de HTML en un taller del W3C. Apple, Mozilla y Opera anuncian continuar trabajando en la mejora en un nuevo foro llamado WHATWG.

Los principios de WHATWG es que las tecnologías deben ser compatibles con versiones anteriores.

2007: W3C forma grupo de trabajo con WHATWG en el desarrollo de HTML5.

2011: Termina la colaboración. Se W3C quería sacar una versión definitiva de HTML5, pero WHATWG quería que sea un estándar de vida.

2019: Whatwg y w3c firman un acuerdo para colaborar en una única versión de HTML en el futuro.

## 2.7 WHATWG

El Grupo de Trabajo sobre Tecnología de Aplicaciones de Hipertexto Web (WHATWG) es una comunidad de personas interesadas en hacer evolucionar la web a través de estándares y pruebas. En 2017, Apple, Google, Microsoft y Mozilla ayudaron a desarrollar una política de propiedad intelectual y una estructura de gobernanza para el WHATWG, y juntos formaron un Grupo Directivo para supervisar las políticas pertinentes.

## 2.8 CONCLUSIÓN

WHATWG es quien mantiene HTML como estándar de vida. W3C mantiene CSS de forma modular.

# 3 CLASE 3: NAVEGADORES

---

## 3.1 JS HISTORIA

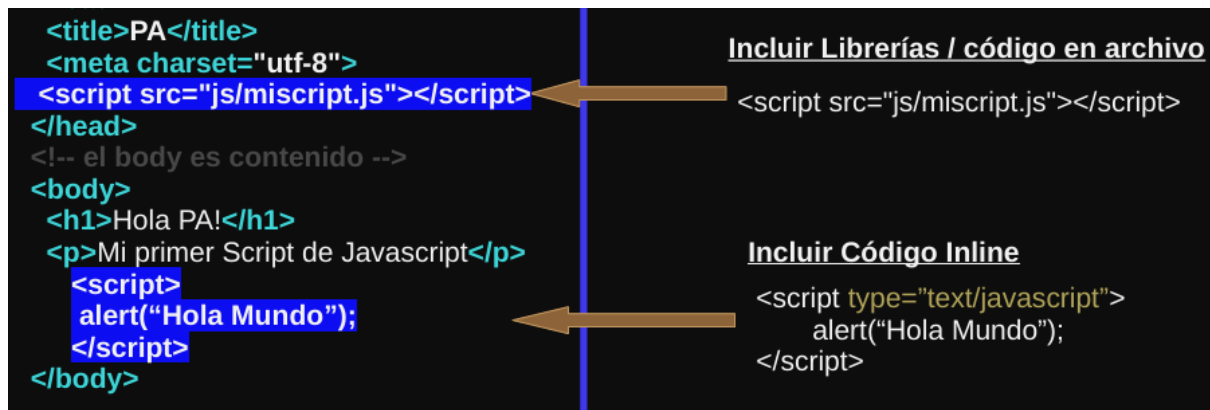
Es un lenguaje de programación interpretado, basado en el estándar ECMAScript. Aunque hoy en día no es tan así, ya que los motores modernos emplean técnicas de compilación Just-In-Time (JIT).

Su uso principal es en el cliente, pero también se puede usar en el servidor gracias a herramientas como Node.js, Electron, o [NW.js](#).

El primer motor de JavaScript fue creado por Brendan Eich en Netscape Communications Corporation, para el navegador web Netscape Navigator en 1996.

Es la tercera capa de tecnologías web estándar.

Como incluirlo:



### 3.2 TIMELINE

1995 / Dic - Sun Microsystems y Netscape anunciaron JavaScript en una conferencia de prensa.

1996 / Marzo - Netscape Navigator 2.0 fue lanzado con soporte para JavaScript.

1997 / Junio - Edición 1 - Lanzamiento del estándar ECMAScript (ECMA-262).

La edición 3 fue abandonada.

Luego anualmente iban sacando una edición, hasta:

2025 - Edición 16 o ES16 Junio 2025.

### 3.3 ECMA

Ecma International es una organización de estandarización dedicada a tecnologías de la información y la comunicación. Su misión es desarrollar y publicar estándares internacionales para la industria de la información y la comunicación.

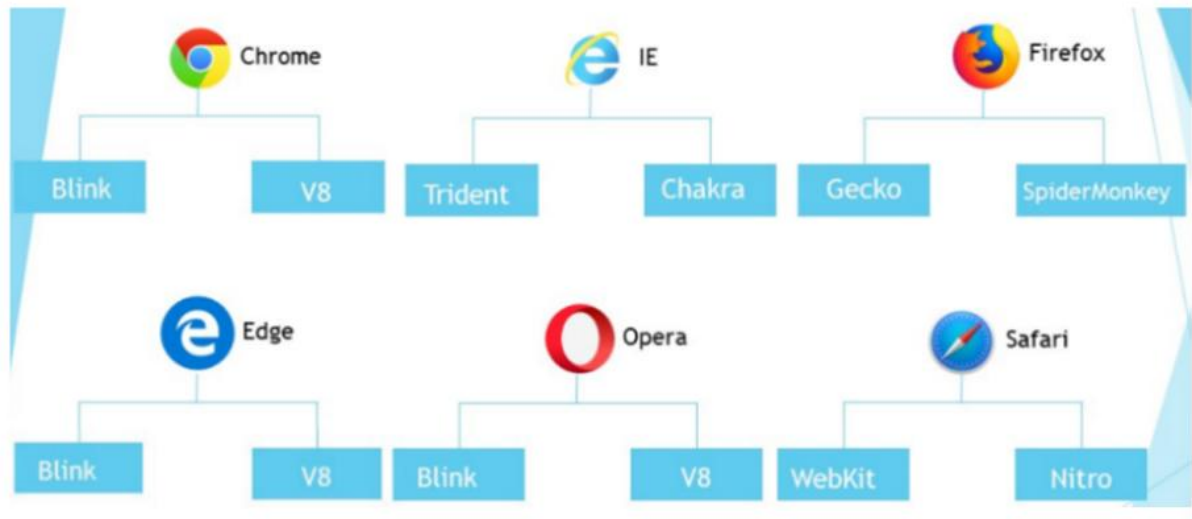
TC39 (Comité Técnico 39) es responsable de la evolución del lenguaje de programación ECMAScript y de la creación de la especificación. El comité opera por consenso y tiene la facultad de modificar la especificación según lo considere oportuno, tiene previsto presentar una especificación a la Asamblea General de la ECMA para su ratificación en julio de cada año. A continuación, se presenta un cronograma para la elaboración de una nueva revisión de la especificación:

- **Febrero:** Se elabora el borrador de candidatos.
- **Febrero-Marzo:** período de exclusión voluntaria sin regalías de 60 días.
- **Reunión TC39 (Marzo):** Se incorporan las propuestas de la Etapa 4, se aprueba la semántica final y se deriva la nueva versión de la especificación. A partir de este momento, solo se aceptan cambios editoriales.
- **Abril-junio:** Período de revisión del Comité Ejecutivo de Ecma y de la Asamblea General (GA) de Ecma.
- **Julio:** Aprobación de la nueva norma por la Asamblea General de ECMA.

### 3.4 JS EN EL CLIENTE

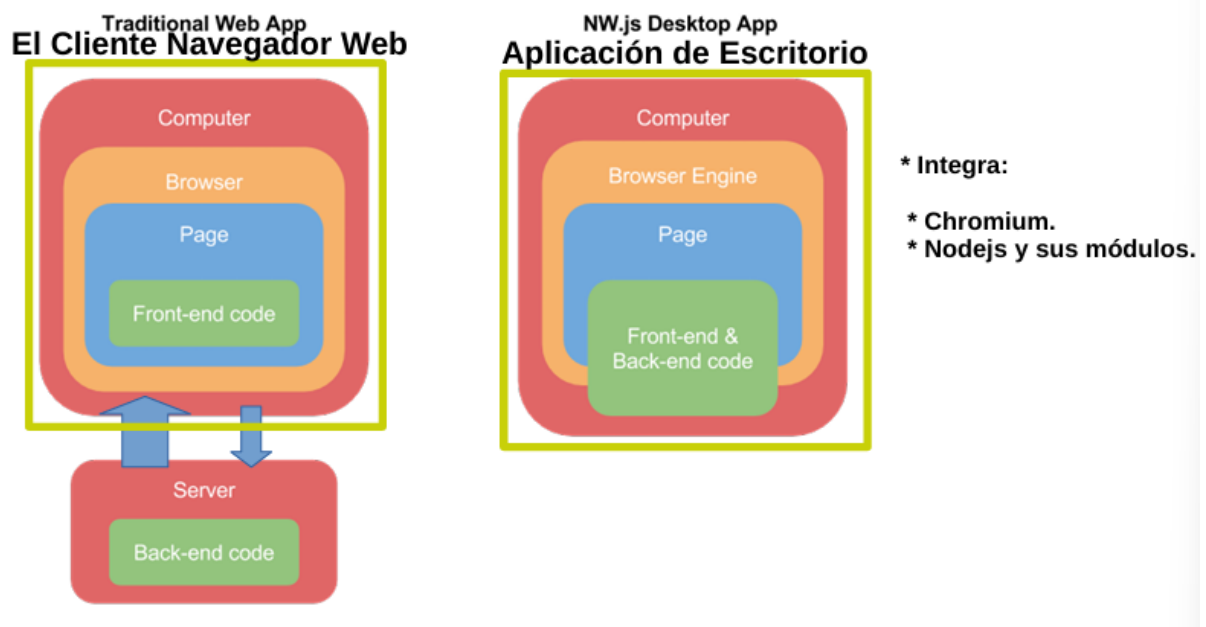
Javascript Engine: Interpretador que entiende y ejecuta código JS. Ejemplos: V8, SpiderMonkey, Chakra, Nitro.

## Motor de Renderizado y Motor de JavaScript



### 3.5 JS EN ESCRITORIO

Existen tecnologías que nos permiten usar JS en el escritorio, como [NW.js](#) o Electron.

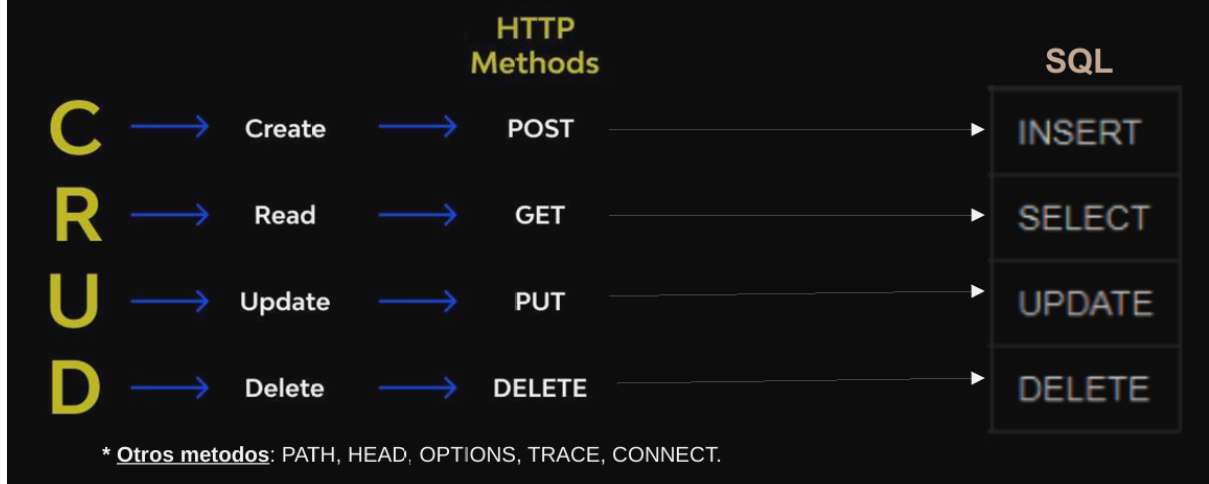


### 3.6 IETF INTERNET ENGINEERING TASK FORCE

Principal organismo de estandarización de Internet, encargado de desarrollar y promover estándares técnicos abiertos que aseguren la interoperabilidad de la red global.

Publican principalmente en forma de RFCs (Request for Comments). Establece las versiones oficiales del protocolo HTTP, como los métodos GET, POST, PUT, DELETE.

Los métodos HTTP, son las operaciones que los clientes pueden realizar sobre los recursos, se encuentran definidos en el estándar IETF → RFC 9110 "HTTP Semantics".



## RFC HTTP - Status Codes



## RFC HTTP - Status Codes



## 4 CLASE 4: IDEs

### 4.1 DEFINICIÓN E IMPORTANCIA DEL IDE

- Un **Entorno de Desarrollo Integrado (IDE)** es una aplicación de software que ayuda a los programadores a desarrollar código de manera eficiente.
- Aumenta la productividad al combinar capacidades como editar, crear, probar y empaquetar software en una aplicación fácil de usar.
- Los IDE proporcionan una interfaz central para herramientas de desarrollo comunes, haciendo que el proceso sea más eficiente, lo que permite a los desarrolladores

comenzar a programar rápidamente sin tener que configurar diferente software manualmente.

## 4.2 CARACTERÍSTICAS Y COMPONENTES PRINCIPALES DE UN IDE

Un IDE generalmente cuenta con las siguientes características:

- **Editor de código fuente:** Un editor de texto con funciones como resaltado de sintaxis, relleno automático específico del lenguaje y comprobación de errores mientras se escribe.
- **Automatización de las compilaciones locales:** Herramientas que automatizan tareas repetitivas como compilar el código fuente a código binario, empaquetarlo y ejecutar pruebas automatizadas.
- **Depurador:** Un programa para probar otros programas y mostrar gráficamente la ubicación de un error en el código original.
- Otras características incluyen:
  - Extensiones.
  - Automatización de la edición de código.
  - Finalización de código inteligente.
  - Integración con control de versiones (Git, SVN).
  - Codificación asistida por IA e Integración con IA.
  - Refactorización.
  - Pruebas Automatizadas.

## 4.3 TIPOS DE IDE

### 4.3.1 IDE locales (o de escritorio):

- Los desarrolladores los instalan y ejecutan directamente en sus equipos locales.
- Son personalizables y no necesitan conexión a Internet una vez instalados.
- **Desafíos:** Pueden consumir mucho tiempo y ser difíciles de configurar, consumir recursos locales, y las diferencias de configuración con el entorno de producción pueden generar errores.

### 4.3.2 IDE en la nube (o Web):

- Permiten a los desarrolladores escribir, editar y compilar código directamente en el navegador, sin necesidad de descargar software en sus equipos locales.
- Ofrecen varias ventajas en comparación con los IDE tradicionales.
- Ejemplos incluyen AWS Cloud9 y Visual Studio Code Web.

## 4.4 ESTADÍSTICAS DE USO DE IDE

- **Encuestas (Stack Overflow 2023):** Visual Studio Code sigue siendo el IDE preferido entre todos los desarrolladores (75.9% en total, 78% entre quienes aprenden a codificar).

- **Google Trends (Agosto 2025): Visual Studio** es el IDE más popular a nivel mundial (28.97% de cuota), seguido por **Visual Studio Code** (15.66%).
- **WakaTime (2024):** Según el tiempo dedicado a la programación, **VS Code** lidera con 33,560,160 horas, seguido por IntelliJ IDEA (4,472,994 hrs).

### Herramientas y Agentes de IA

- Existen herramientas y extensiones de IA para el desarrollo, especialmente en Visual Studio Code:
  - **GitHub Copilot:** Asistente de IA para autocompletar código.
  - **Copilot Chat:** Interacción conversacional con IA.
  - **CodeGPT:** Extensión para usar modelos LLM en VS Code.
  - **Cursor:** Editor con reglas y memorias inteligentes.

## 5 CLASE 5: UX Y UI

### 5.1 DEFINICIÓN DE UX Y UI

- **UX (User Experience):** Experiencia de Usuario.
- **UI (User Interface):** Interfaz de Usuario.
- **Relación UX/UI:** La UX es una disciplina que se interrelaciona con otras, incluyendo la UI. Existe una jerarquía desde lo más Abstracto (Diseño de Experiencias - UX) hasta lo más Concreto (Diseño de Visual - VD).
  - La UI se centra en la apariencia y el estilo.
  - El Diseño Visual, el Diseño de la Interfaz y el Diseño de la Navegación son capas concretas de los Elementos de la Experiencia de Usuario de Garret (2002).

### 5.2 DISCIPLINAS RELACIONADAS Y ENFOQUE

**Disciplinas Interrelacionadas con UX:** Marketing, Diseño Visual, Investigación de usuario, Accesibilidad, Antropología, Arquitectura de la información, Usabilidad, Psicología, Sociología, Diseño de interacción, Gestión de Contenidos, Diseño industrial, Análisis de negocio y Gestión de proyectos.

UX > IxD > UI > VD (XIIIV) para memorizar

#### 5.2.1 Design Thinking: Un proceso iterativo de 5 fases:

1. **Empatizar:** Comprender profundamente las necesidades y problemas del usuario.
2. **Definir:** Sintetizar la información para identificar el problema clave a resolver.
3. **Idear:** Generar ideas y explorar alternativas para solucionar el problema.
4. **Prototipar:** Desarrollar prototipos de las ideas más prometedoras para visualizarlas y probarlas.



5. **Probar:** Evaluar los prototipos con usuarios reales para recibir retroalimentación y mejorar la solución.

**UX Research:** Es el rol de investigador, el que testea y el que investiga la problemática, generando las bases. Los métodos de investigación incluyen entrevistas, encuestas y análisis del comportamiento de los usuarios.

### 5.3 DISEÑO DE INTERFAZ (UI)

El **diseñador UI** crea visualmente la interfaz del producto conforme a la experiencia de usuario para proporcionar una experiencia intuitiva y diferenciadora. La experiencia intuitiva se refiere a que sea simple para los usuarios y se deben simplificar los componentes o elementos en la interfaz.

**Tipos de Interfaz:**

- **GUI (Interfaces Gráficas de Usuario):** Los usuarios interactúan con representaciones visuales.
- **VUI (Voice User Interface):** Los usuarios interactúan por voces (e.g., Siri, Alexa).
- **3DUI:** Espacios de diseño 3D a través de movimientos corporales.

### 5.4 ELEMENTOS Y PRINCIPIOS DEL DISEÑO VISUAL

El Diseño Visual es la suma de **Elementos de Diseño Visual** y **Principios de Diseño**.

#### 5.4.1 Elementos del Diseño Visual (7):

- **Línea:** Puede ser gruesa, delgada, recta, curva, etc., y puede transmitir diferentes emociones.
- **Forma:** Áreas autónomas (dos dimensiones: ancho y largo), generalmente formadas por líneas, aunque también por color, valor o textura.
- **Espacio Negativo/Positivo:** El espacio negativo (espacio en blanco) es el área vacía alrededor de una forma positiva. La relación entre ambos se llama Figura/Fondo.
- **Volumen:** Elementos visuales tridimensionales (largo, ancho y profundidad). En diseño digital, se simula con sombras, degradados y perspectivas.
- **Valor:** Describe la luminosidad (claro) y la oscuridad (oscuro).
- **Color:** Para pantallas se usa **RGB** (colores aditivos: Rojo, Verde, Azul), y para impresión **CMYK** (colores sustractivos: Cian, Magenta, Amarillo, Black).
- **Textura:** Puede ser táctil o implícita.

#### 5.4.2 Principios del Diseño (6):

- **Unidad:** Transmitir un mensaje de forma coherente y armoniosa; los elementos cercanos se perciben como un mismo grupo.
- **Gestalt:** La mente percibe las cosas como un todo, no individualmente. Sus principios incluyen Proximidad, Cierre, Igualdad, Figura-Fondo, Dirección y Continuidad.
  - **Proximidad:** Elementos cercanos se perciben como una unidad.
  - **Cierre:** Objetos incompletos se perciben como completos.
  - **Igualdad:** Elementos similares (tamaño, forma, color) se agrupan visualmente.



- **Figura y fondo:** La mente organiza la información en figuras que se destacan del fondo<sup>39</sup>.
- **Dirección:** Las líneas guían la mirada.
- **Continuidad:** La mente sigue una dirección o patrón continuo.
- **Simetría:** Las formas simétricas se perciben como más agradables y equilibradas.
- **Jerarquía:** Muestra la diferencia de importancia de los elementos en un diseño, siendo el color y el tamaño las formas más comunes para crearla<sup>43434343</sup>.
- **Balance (Equilibrio):** Distribución uniforme de los elementos para lograr un diseño tranquilo y estable.
- **Contraste:** Se usa para destacar un elemento mediante el uso del color, el valor y el tamaño.
- **Escala:** Describe los tamaños relativos de los elementos para enfatizar uno sobre otros.

## 5.5 PROTOTIPO

- **Prototipo de baja fidelidad (Low-Fi):** Representación inicial y simplificada de una idea. Se hace en papel o herramientas básicas. Busca validar estructuras y flujos de navegación de forma rápida y económica.
- **Prototipo de alta fidelidad (High-Fi):** (Se muestra un ejemplo de diseño con detalles visuales, implicando que son más cercanos al producto final).

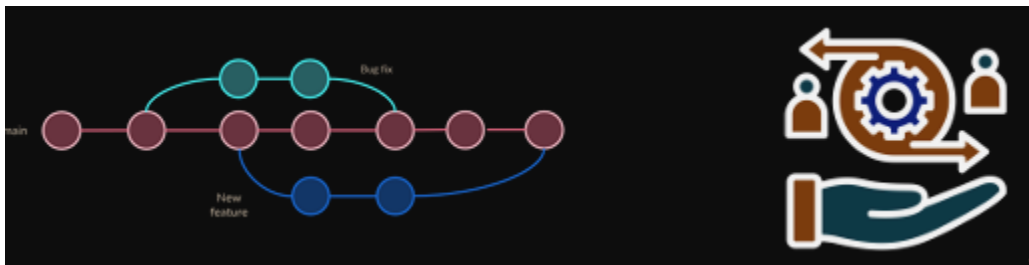
## 6 CLASE 6: CONTROL DE VERSIONES

---

### 6.1 CONCEPTOS FUNDAMENTALES DEL CONTROL DE VERSIONES

#### ¿Qué es un Sistema de Control de Versiones (VCS)?

- Es una herramienta utilizada en el desarrollo de software y la gestión de proyectos para rastrear y administrar los cambios en los archivos a lo largo del tiempo.
- Registra los cambios en un archivo o conjunto de archivos a lo largo del tiempo, permitiendo recuperar versiones específicas posteriormente.
- Permite revertir archivos seleccionados o todo el proyecto a un estado anterior, comparar cambios, ver quién introdujo un problema y cuándo, y facilita la recuperación de archivos perdidos.



#### 6.1.1 Puntos Clave de un VCS

- **Seguimiento de Cambios:** Registra cada modificación, incluyendo quién la hizo, cuándo y qué se modificó, proporcionando trazabilidad.

- **Colaboración:** Permite a los equipos trabajar en paralelo y fusionar/coordinar los cambios de manera sencilla.
- **Control de Versiones:** Mantiene múltiples versiones, facilitando la recuperación de estados anteriores.
- **Resolución de Conflictos:** Ofrece herramientas para resolver conflictos que surgen al fusionar cambios hechos por varios usuarios en el mismo archivo.
- **Ramificación (Branching) y Fusión (Merging):** Permite crear copias separadas del código principal (ramas) para trabajar en nuevas características sin afectar la versión principal, y luego fusionarlas.
- **Respaldo:** Al usar un sistema remoto, los archivos y cambios están respaldados contra pérdidas accidentales.

## 6.2 TIPOS DE SISTEMAS DE CONTROL DE VERSIONES

### 6.2.1 Sistemas de Control de Versiones Centralizados (CVCS)

- Existe un único repositorio central con todas las versiones.
- Los usuarios trabajan en una copia y deben enviar los cambios al repositorio central para que sean accesibles a otros.
- Ejemplos: SVN (Subversion) y CVS (Concurrent Versions System).

### 6.2.2 Sistemas de Control de Versiones Distribuidos (DVCS)

- Cada usuario tiene su propio repositorio local con una copia completa del historial de versiones.
- Permite trabajar de manera independiente sin conexión constante al repositorio central.
- Los cambios se sincronizan entre repositorios locales y remotos.
- Ejemplos: **Git** y Mercurial.

## 6.3 GIT Y PLATAFORMAS COLABORATIVAS

**Git:** Es un sistema de control de versiones distribuido, gratuito y de código abierto, diseñado para gestionar proyectos de cualquier tamaño con velocidad y eficiencia.

### Características Principales (4):

- Distribuido (copia completa del repositorio local)
- Rápido y eficiente (almacena como instantáneas)
- Ramificación/fusión eficiente.
- Utiliza un sistema de hash SHA-1 para la integridad de los datos.

#### 6.3.1 Historia:

2005: Linus Torvalds anuncia que está laburando en git. No estaba satisfecho con las herramientas de control de versiones que había.

2007: Git se desarrolla y se utiliza internamente para administrar el código fuente del kernel de Linux.

2008: Se lanza github para alojar código con git, y lo hace popular.

2010: Git se convierte en el SCV por defecto del kernel.

### 6.3.1.1 Git vs. GitHub

| <br>GITHUB | VS | <br>GIT |
|---------------------------------------------------------------------------------------------|----|------------------------------------------------------------------------------------------|
| 1 GitHub is a service                                                                       |    | 1 Git is a software                                                                      |
| 2 GitHub is a graphical user interface                                                      |    | 2 Git is a command-line tool                                                             |
| 3 GitHub is hosted on the web                                                               |    | 3 Git is installed locally on the system                                                 |
| 4 GitHub is maintained by Microsoft                                                         |    | 4 Git is maintained by linux                                                             |
| 5 GitHub is focused on centralized source code hosting                                      |    | 5 Git is focused on version control and code sharing                                     |
| 6 GitHub is a hosting service for Git repositories                                          |    | 6 Git is a version control system to manage source code history                          |

### Otras Plataformas Colaborativas

- Existen servicios en la nube para Git, como **GitLab**, **Bitbucket** y **Gitea**<sup>39</sup>.
- **Gitea**: Es una plataforma de gestión de repositorios Git autoalojada, diseñada para ser ligera, rápida y fácil de instalar. Desarrollada por Gugler.

## 6.4 FLUJO BÁSICO Y CLIENTES DE GIT

### 6.4.1 Flujo de Trabajo (Áreas de Git)

1. **Workspace** (Área de trabajo)
2. **Staging** (Área de ensayo)
3. **Local Repository** (Repositorio local)
4. **Remote Repository** (Repositorio remoto)
  - Comandos: `git add/mv/rm` (mueve del *workspace* al *staging*), `git commit` (mueve del *staging* al *local repository*), `git push` (mueve del *local* al *remote repository*).

### 6.4.2 Clientes GUI e Integración con IDE

Existen herramientas de terceros con Interfaz Gráfica de Usuario (GUI) para Git, como GitHub Desktop, SourceTree y TortoiseGit.

Muchos Entornos de Desarrollo Integrado (IDE) incluyen soporte nativo de Git, permitiendo realizar operaciones de control de versiones directamente desde el editor (como se muestra en una captura de VS Code).

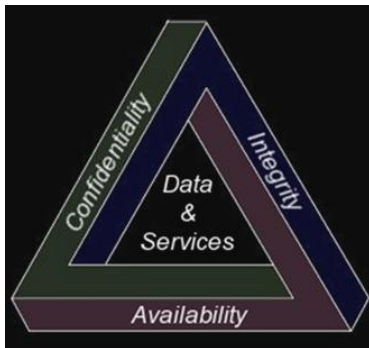
## 7 CLASE 7: SEGURIDAD EN APPS WEB

---

### 7.1 FUNDAMENTOS DE LA SEGURIDAD (TRIÁNGULO CIA)

La seguridad en un sistema implica garantizar tres aspectos principales:

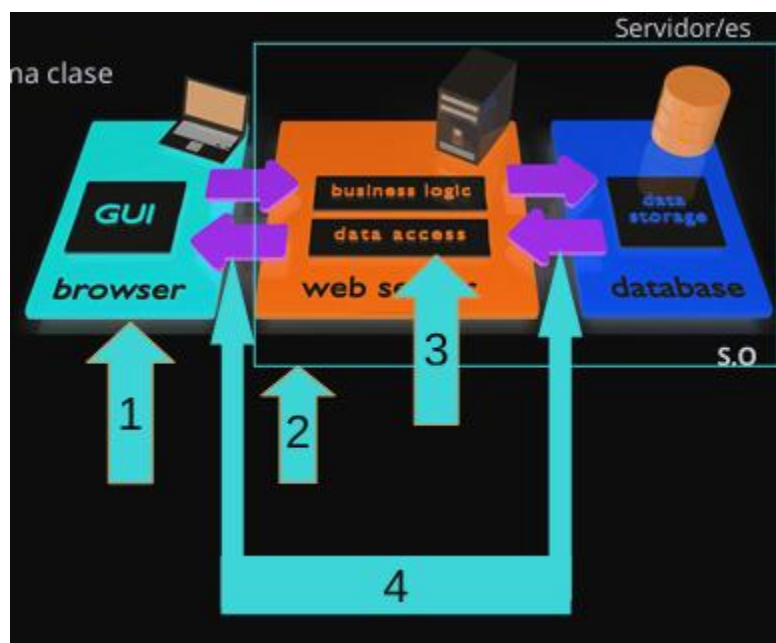
- **Confidencialidad:** Solo los elementos autorizados pueden acceder a los objetos del sistema.
- **Integridad:** Solo los elementos autorizados pueden modificar los objetos del sistema.
- **Disponibilidad:** Los objetos del sistema deben permanecer accesibles y utilizables para los elementos autorizados cuando se requiera.



### 7.2 RIESGOS Y PUNTO DE EQUILIBRIO

- **Riesgos:** Los atacantes pueden usar diferentes rutas a través de la aplicación para causar daño. El impacto de un ataque puede ser técnico (afectando a recursos o funciones) y al negocio.
- **Riesgo Cero:** El riesgo cero no existe. Las opciones para manejar el riesgo son: Evitar/Eliminar, Reducir/Mitigar, Asumir/Aceptar o Transferir/Compartir.
- **Costo/Seguridad/Riesgo:** Debe evaluarse el costo. No se debe gastar un millón para proteger un céntimo, buscando un **Punto de Equilibrio** entre el nivel de seguridad y el costo asociado.

### 7.3 ASPECTOS DE SEGURIDAD EN UNA APLICACIÓN WEB



Hay 4 aspectos de seguridad a considerar al diseñar una aplicación web:

1. **Seguridad en el Cliente:** Consiste en aplicar una capa de seguridad en el cliente (navegador) para ahorrar trabajo en el servidor y dar interactividad. No debe confiarse en ella, ya que se puede puentear con facilidad.
2. **Seguridad en el Servidor:** Seguridad propia del servidor y sus servicios (web, aplicaciones, bases de datos). Los problemas surgen por versiones no actualizadas, configuraciones por defecto inadecuadas o cuentas por defecto activas. Una política de *backups* también es parte de la seguridad.
3. **Seguridad en la Aplicación:** Seguridad establecida en la aplicación en sí. Engloba:
  - **Controles de Acceso (AAA o AAR en español):** Restringir el acceso a zonas limitadas. Incluye: **Autenticación** (determinar si un usuario es quien dice ser), **Autorización** (comprobar si tiene permiso para una acción), y **Registro** (capacidad de registrar eventos).
    - Los *passwords* deben seguir reglas de complejidad, almacenarse de forma segura (protegiendo el acceso a la base de datos), y las cuentas deben bloquearse tras intentos incorrectos.
    - La **gestión de sesiones** debe limitar el tiempo de vida, regenerar identificadores, detectar ataques de fuerza bruta, requerir nueva autenticación para operaciones importantes y proteger los identificadores durante la transmisión.
  - **Validación de datos:** Es el problema más frecuente y la causa de vulnerabilidades como Inyección SQL y Cross-Site Scripting (XSS). Las fuentes de entrada (URL, Cookies, Cabeceras HTTP, Campos de formularios) pueden ser manipuladas por el usuario.
    - **Estrategia de Protección:** La más adecuada es **Aceptar únicamente datos válidos conocidos** (solo aceptar datos que se adapten a lo esperado). Otras son Rechazar datos no válidos conocidos o Sanear todos los datos.

- **Programación Segura:** Seguir buenas prácticas de programación. Esto incluye inicializar variables, una gestión adecuada de errores (los mensajes de error dan información sensible a atacantes) , y proteger la información sensible fuera del árbol de directorios web. El **testing de seguridad** debe ser integral, no una etapa posterior.
  - **Fuentes de entrada:** Cadenas URL, Cookies, Cabeceras HTTP y Campos de formularios.
4. **Seguridad en la Comunicación:** (Mencionado en el diagrama, pero sin detalle en el texto).

## 7.4 CIFRADO VS. HASH

- **Cifrado:** Mantiene en secreto el contenido para que pueda ser descifrado con una clave secreta (Confidencialidad). Puede ser **Simétrico** (DES, Triple-Des, Blowfish, AES) o **Asimétrico** (RSA, DSA, ElGamal).
- **Hash:** Valida que el contenido no ha sido modificado sin necesidad de ver el contenido (Integridad).

## 7.5 OWASPTOP 10

**OWASP** (Open Web Application Security Project): Fundación dedicada a combatir las causas del software inseguro. Publica un documento (Top 10) sobre las **vulnerabilidades más críticas** en aplicaciones web.

## 7.6 RECOMENDACIONES DE ALTO NIVEL

Es conveniente:

- Validar la entrada y sanear las salidas. (*"Todos mienten" – Dr. House*).
- Mantener un esquema de seguridad simple.
- Utilizar componentes de confianza.
- Mantener los privilegios al mínimo y separados.
- La seguridad por oscuridad no funciona.

# 8 CLASE 8: LARAVEL

---

## 8.1 QUÉ ES

Framework de desarrollo web basado en PHP, que sigue la arquitectura MVC. Ofrece herramientas y características avanzadas como enrutamiento, migraciones de BDs, autenticación de colas y más. Promueve buenas prácticas como Eloquent ORM para interactuar con las BDs y inyección de dependencias para un código más modular y reutilizable.

Laravel esta basado en Symphony, cuando salió llevo a PHP a otro nivel.

## 8.2 CARACTERÍSTICAS

- **Sintaxis Expresiva y Elegante**
- **ORM Eloquent**

- **Migraciones a Base de Datos:** Para facilitar la gestión de esquemas, las migraciones permiten versionar los cambios en la estructura de la base de datos.
- **Sistema de Enrutamiento sencillo:** Laravel permite control granular de rutas.
- **Colas y Tareas en Segundo plano:** Maneja trabajos en cola, y tareas en segundo plano para mejorar el rendimiento.
- **Testing Integrado:** Soporte para pruebas unitarias e integradas.
- **Comunidad y Ecosistema Amplio:** Con herramientas como:
  - **Laravel Forge:** Para implementar servidores.
  - **Laravel Vapor:** Desplegar apps en AWS sin servidor.
  - **Laravel Horizon:** Monitorear colas de trabajos.
  - **Laravel Nova:** Panel administrativo avanzado.
- Soporte para Eventos y Broadcasting: Característica avanzada que muchos frameworks no proveen.
- Posee un derivado llamado **Lumen**, orientado a la creación de APIs y Micro-Services.

### 8.3 ESTRUCTURA

#### Estructura

La estructura de directorios principales es la siguiente:

**app/:** Contiene el código de la aplicación, como controladores, modelos, políticas y servicios. Es el núcleo lógico de la aplicación.

**bootstrap/:** Contiene el archivo app.php que inicia el framework. También almacena la caché generada al optimizar el framework.

**config/:** Aquí están los archivos de configuración para diferentes partes del framework, como base de datos, correo, sesión, etc.

**database/:** Contiene las migraciones, fábricas y seeds de la base de datos.

**public/:** Es el único directorio accesible públicamente, contiene index.php (puerta de entrada a la aplicación), assets como imágenes, CSS y JavaScript.

**resources/:** Aquí se almacenan las vistas Blade, archivos de lenguaje y assets sin compilar (SASS, JavaScript).

**routes/:** Almacena los archivos de rutas web y de API (web.php, api.php).

**storage/:** Contiene archivos generados por la aplicación, como logs, caché y archivos subidos.

**tests/:** Directorio para escribir y ejecutar pruebas automatizadas.

**vendor/:** Almacena todas las dependencias del proyecto que se manejan con Composer.

#### 8.3.1 MVC:

Lo visto en la práctica. Modelo, Vista, Controlador.

Luego se habla de la Capa de Presentacion, que seria la Vista, Capa de Negocio, que seria el Controlador y Capa de Acceso a Datos que serian los Modelos.

**La idea es separar responsabilidades y reutilizar código.**

### 8.4 FUNCIONAMIENTO

1. TODAS las peticiones HTTP entran por el archivo public/index.php que hace de punto de entrada a la app: Aca arranca el ciclo de vida de Laravel.
2. De aca comprueba que existan las rutas en los archivos routes/web.php o routes/api.php dependiendo la petición.
3. Si existe una ruta, se identifica el controlador y el método que se encargará de manejarla.
4. Se asigna la petición al método del controlador asociado a la ruta.

## 8.5 LARAVEL COMO FRAMEWORK

- Esta en la versión 11.0
- Blade es el motor de plantillas utilizado por Laravel.
- Integra ORM (Mapea tablas de la BD a objetos en PHP) llamado Eloquent.
- Sistema de Ruteo y también RESTfull.
- Basado en Composer.
- Soporte para Cache.
- **Artisan**: es quien le da una interfaz de línea de comandos para poder utilizarse.
  - Se invoca con: *php artisan <comando>*
- Muy utilizado.

### 8.5.1 Instalación

- Requerimos un servidor web como Apache o ngnix.
- Requerimos al menos un motor de BD como Mysql, Postgre, SQLite, SQL Server, etc.
- Puede usar soluciones como XAMPP o servicios en Docker.
- Requiere tener Composer instalador de paquetes de PHP.
- Requiere librerías como **mcrypt, mbstring, xml, pdo**.

SPA: Single Page Application.

8.5.2 Para crear un proyecto con laravel es:

*Composer create/project Laravel/Laravel nombre-tu-proyecto*

Automaticamente te baja librerías como Guzzle, Monolog, Carbon, y Vite.

**Iniciar servidor de desarrollo local:**

*\$ php artisan serve http://localhost:8000*

### 8.5.3 .env

El archivo de configuración de entorno .env no debe ser versionado. Contiene valores específicos de tu entorno de desarrollo. Parámetros comunes:

APP\_NAME, APP\_URL, APP\_KEY (cadena única que genera laravel), APP\_DEBUG (bool, determina si se muestran o no errores en el navegador), y parámetros para la DB.

## 8.6 ARTISAN - COMANDOS

### 8.6.1 Comandos de servidor y mantenimiento

*\$ php artisan serve*

*\$ php artisan down* pone la app en modo mantenimiento, y pone la página de “pronto volvemos”.

*\$ php artisan up* saca la app de modo mant

*\$ php artisan list* lista los comandos

### 8.6.2 Comandos de Generación (Scaffolding)

Comandos esenciales para crear estructura de la app de forma rápida usando las convenciones de Laravel.

*Php artisan make: model <NombreModelo>* crea un nuevo modelo de Eloquent.

-m o -migration: crea una migración para el modelo.

-c o -controller: crea un controlador para el modelo

-a o -all: crea migración, controlador y el factory (para datos de prueba).



## Artisan

`php artisan make:controller <NombreControlador>`: Crea un nuevo controlador.

Opciones comunes:

`--resource`: Crea un controlador con métodos para operaciones CRUD (index, create, store, show, edit, update, destroy).

`--api`: Similar a `--resource`, pero solo genera los métodos que se usarían en una API (index, store, show, update, destroy).

`php artisan make:migration create_usuarios_table`: Crea un nuevo archivo de migración de base de datos.

`php artisan make:seeder <NombreSeeder>`: Crea una clase para "sembrar" la base de datos con datos de prueba.

`php artisan make:factory <NombreFactory>`: Crea un "factory" para generar modelos con datos ficticios.

`php artisan make:request <NombreRequest>`: Crea una clase para validar los datos de una solicitud HTTP.

## 8.7 VISTAS Y BLADE

No se recomienda realizar consultas ni procesamiento de datos en las mismas. Solo reciben datos y los preparan para mostrarlos como HTML. Se ubican en `resources/views/`.

Vistas – Estructuras condicionales

```
@if (condicion)
|   #Logica a resolver
@endif

@if (condicion)
|   #Logica a resolver
|   #Logica a resolver
|   #Logica a resolver
@endif

@for ($i = 0; $i < 10; $i++)
|   #El valor actual es {{ $i }}
@endfor

@while (true)
|   <p>Soy un bucle while infinito!</p>
@endwhile

@foreach ($users as $user)
|   <p>Usuario {{ $user->name }} con identificador:</p>
@endforeach
```

Los @ son las plantillas de Blade.

### 8.7.1 Subvistas

`@include` de Blade permite incluir una vista de Blade en otra vista. Las variables de la vista padre estarán disponibles para la vista incluida. Además de heredar estos datos, se le puede pasar un array de datos adicional.

```
@include('view.name', ['status' => 'complete'])
```

## 8.7.2 Layout Blade

### Layout Blade

```
mi_vista_hija.blade.php (Vista Hija)

HTML

@extends('layouts.master')

@section('menu')
    @parent
    <p>Este contenido se añade al menú principal.</p>
@endsection

@section('content')
    <p>Esta sección aparecerá en la sección "content".</p>
@endsection
```

**@extends('layouts.master')**: Esta directiva le dice a una vista hija que debe heredar su estructura de un archivo de layout padre, en este caso, `resources/views/layouts/master.blade.php`.

**@yield('content')**: Esta directiva, ubicada en el layout padre, funciona como un marcador de posición. Define una sección llamada `content` donde las vistas hijas pueden inyectar su contenido específico.

**@section('content') ... @endsection**: Esta directiva, utilizada en la vista hija, define el contenido que se colocará en la sección `yield` del mismo nombre en el layout padre.

**@parent**: Esta directiva permite incluir el contenido de la sección del layout padre antes o después del contenido de la vista hija. A menudo se usa para añadir algo a una sección en lugar de sobrescribirla por completo.

## 8 CLASE 9: SELENIUM

---

## 9 CLASE 10: LARAVEL 2: LA RESURRECCIÓN

---

### 9.1 CONFIGURACIÓN DE LA BASE DE DATOS

Laravel soporta diferentes tipos de bases de datos como MySQL, PostgreSQL, SQLite y SQL Server.

La configuración se realiza principalmente en el archivo `config/database.php` y las credenciales se recomiendan establecer en el archivo oculto `.env` para evitar exponer claves en repositorios de código.

Se pueden usar varias bases de datos simultáneamente, incluso de distinto tipo, indicando la conexión por defecto y especificando otras al consultarlas.

Para PostgreSQL, se mencionan variables de configuración como `DB_CONNECTION=pgsql`, `DB_HOST`, `DB_PORT=5432`, `DB_DATABASE`, `DB_USERNAME` y `DB_PASSWORD`.

### 9.2 MAPEO OBJETO-RELACIONAL (ORM) - ELOQUENT

- **ORM** (Object Relational Mapping) es una técnica para convertir datos entre un lenguaje orientado a objetos y una base de datos relacional.
- **Eloquent** es el ORM propio de Laravel, que proporciona una forma fácil de interactuar con la base de datos. **No es compatible con bases de datos NoSQL.**
- **Modelos**: Una clase que representa una tabla en la base de datos y se usa para interactuar con ella de forma orientada a objetos (operaciones CRUD: Crear, Leer, Actualizar, Eliminar). Los modelos se almacenan en `app/Models` y extienden la clase `Illuminate\Database\Eloquent\Model`.
- **Convenciones de Eloquent**:

- **Nombre de la tabla:** Por defecto, el nombre del modelo es singular con mayúscula inicial (ejemplo: Product), y se mapea a una tabla en plural y minúsculas (ejemplo: products).  
Se puede sobrescribir usando la propiedad `protected $table`.
- **Clave Primaria:** Se asume que es id. Se puede cambiar con `protected $primaryKey`.
- **Timestamps:** Por defecto, se asume que las tablas tienen los campos `updated_at` y `created_at`. Se actualizan automáticamente. Para desactivarlos, se usa `public $timestamps = false`.

### 9.3 MIGRACIONES DE BASE DE DATOS

- Las migraciones controlan las versiones del esquema de la base de datos a través de archivos PHP en lugar de instrucciones SQL directas.
- **Funcionamiento:**
  - Se crean con el comando de Artisan `php artisan make:migration`.
  - Cada archivo contiene los métodos `up()` (para aplicar los cambios, como crear una tabla o columna) y `down()` (para revertir los cambios).
  - Se ejecutan con `php artisan migrate`.
  - Se puede deshacer la última migración con `php artisan migrate:rollback`.
  - `php artisan migrate:refresh` revierte todos los cambios y vuelve a aplicar las migraciones.
- **Ventajas:** Colaboración en equipo, control de versiones, y consistencia en los entornos.
- **Schema Builder:** Permite definir columnas, tipos de datos (`$table->string('name')`, `$table->integer('votos')`, `$table->timestamps()`, etc.) y modificadores (`->nullable()`, `->default($value)`, etc.). También soporta la creación de **índices** (`->unique()`, `->index()`, etc.) y **restricciones de clave foránea**.

### 9.4 INICIALIZACIÓN DE LA BASE DE DATOS (DATABASE SEEDING)

- Facilita la inserción de datos iniciales o datos de prueba.
- Los archivos "semilla" están en `database/seeds`.
- La clase principal es `DatabaseSeeder`, cuyo método `run()` se llama al iniciar.
- Se pueden crear clases *seeder* modulares (ejemplo: `UsersTableSeeder`) con `php artisan make:seeder NombreSeeder` y se llaman desde el método `run()` de `DatabaseSeeder` con `$this->call(ClaseSeeder::class)`.
- Se ejecutan con el comando de Artisan `php artisan db:seed`

### 9.5 CONSTRUCTOR DE CONSULTAS (QUERY BUILDER)

- Es una herramienta que facilita la interacción con la base de datos usando una interfaz fluida en PHP, previniendo la inyección de SQL.
- Permite realizar operaciones básicas como **SELECT** (`DB::table('users')->get()`), **INSERT** (`->insert()`), **UPDATE** (`->where()->update()`) y **DELETE** (`->where()->delete()`).

- Soporta cláusulas **WHERE**, **JOIN** y **funciones de agregación** como `count()`, `avg()`, `max()`, etc.

## 9.6 RELACIONES ENTRE MODELOS CON ELOQUENT ORM

Eloquent soporta varias relaciones entre modelos:

- **Uno a Uno (1 a 1):** Un registro se relaciona solo con otro, y viceversa (ejemplo: User tiene un Profile). Se usa `hasOne()` y `belongsTo()`.
- **Uno a Muchos (1 a n):** Un registro tiene muchos registros de otro modelo, pero el segundo modelo pertenece a un solo registro del primero (ejemplo: User tiene muchos Post) <sup>454545454545454545</sup>. Se usa `hasMany()` y `belongsTo()`.
- **Muchos a Muchos (n a m):** Cada registro puede estar relacionado con varios registros, y estos a su vez con otros (ejemplo: Student en muchos Course, y Course con muchos Student). Requiere una tabla intermedia. Se usa `belongsToMany()`.
- **Relaciones de pertenencia a través de otras entidades:** Permite definir una relación directa entre dos entidades pasando de manera transparente a través de una tercera.
- **Relaciones Polimórficas:** Permite que un modelo pertenezca a más de un tipo de modelo usando una única tabla intermedia (útil para evitar duplicación de tablas, como una sola tabla de comments que referencia a Student o Teacher).

## 10 CLASE 11: SEGURIDAD WEB PARTE 2: AHORA ES PERSONAL

---

### 10.1 SEGURIDAD EN LA COMUNICACIÓN Y CIFRADO DE INFORMACIÓN

El tema central es la Seguridad en la Comunicación y el Cifrado de la información utilizando tecnologías como SSL/TLS, Let's Encrypt y Certificados.

### 10.2 ARQUITECTURA DE APLICACIONES WEB (GENERAL)

Una aplicación web típicamente involucra un navegador (cliente) que interactúa con un servidor web, que a su vez accede a una base de datos.

Cliente -> Server -> BD

### 10.3 HTTP vs. HTTPS

#### 10.3.1.1 HTTP (Hypertext Transfer Protocol)

Funcionamiento: Un navegador (cliente) envía un mensaje de petición al servidor, y el servidor devuelve un mensaje de respuesta (encabezado con tipo de contenido y el documento). Los mensajes consisten en un conjunto de líneas de texto.

Debilidades:

- Tanto las peticiones como las respuestas viajan como cadenas de texto plano.
- Esto presenta vulnerabilidades de seguridad.
- Ataques comunes en HTTP: Intruso Man-in-the-Middle (MitM) en LAN o WAN (*Sniffer*), Fuerza Bruta y Phishing.
- La información sensible, como los datos de una tarjeta de crédito, puede viajar en texto plano y ser interceptada (vulnerabilidades de Man in the Middle o Sniffer).

### 10.3.1.2 HTTPS

- Propósito: Proporciona confidencialidad y autenticación en la web.
- Funcionamiento: Cifra la comunicación entre el servidor y el cliente usando claves y algoritmos de cifrado.
- Protocolos de Cifrado: El cifrado en HTTPS se realiza mediante algoritmos de clave simétrica. Utiliza los protocolos SSL (Secure Sockets Layer) y su sucesor, TLS (Transport Layer Security).

## 10.4 EVOLUCIÓN DE SSL/TLS

- SSL (Secure Sockets Layer): Creado por Netscape e introducido en su navegador en 1995.
  - La primera versión de SSL (SSLv2) fue introducida por Netscape en 1995.
  - Netscape luego creó una mejora, SSLv3, publicada en 1996.
- TLS (Transport Layer Security):
  - Introducido en 1999 por la IETF (Grupo de Trabajo de Ingeniería de Internet) como un cuarto protocolo similar, denominado TLSv1.0, ante la existencia de estándares similares pero incompatibles (SSL y PCT de Microsoft).
  - Posteriores versiones: TLS v1.1 (2006) y TLS v1.2 (2008).
  - La versión más reciente es TLS v1.3, definida en el RFC 8446 en 2018.
- Mejoras en Versiones: Cada actualización mejora los algoritmos de las *cipher suites* (suites de cifrado) de conexión y cifrado, así como la compatibilidad.
- Diferencias Clave TLS 1.2 vs. TLS 1.3: TLS 1.3 ofrece mejoras en la velocidad de enlace (*handshake*) y la seguridad, requiriendo menos viajes de ida y vuelta que TLS 1.2, utilizando algoritmos criptográficos más rápidos, y eliminando cifrados inseguros.

### 10.4.1 Certificados Digitales (SSL/TLS)

#### Certificados de Servidor

- Definición: Un documento electrónico bajo el estándar X.509, emitido por una Autoridad Certificadora (CA).
- Propósito:
  1. Autenticar la identidad de un servidor.
  2. Cifrar la comunicación entre el servidor y el cliente.
- Contenido: Consta de un par de claves: una pública (incluida en el certificado) y una privada.
- Validación (Cadena de Confianza): Los clientes (navegadores) verifican el certificado utilizando una cadena de confianza.
  - Certificado Raíz (CA): Autofirmado, contiene el Nombre CA y la Clave Pública.
  - Certificado Intermedio: Firmado por el CA Raíz, contiene la Clave Pública Intermedia y el Nombre del Emisor (CA Raíz).
  - Certificado Final/Servidor: Firmado por una CA o CA Intermedia, contiene la Clave Pública Final y el Emisor (Intermedio).

- Almacenamiento de CAs: Los Certificados Raíz y CA Intermedios se encuentran en los almacenes de certificados, incorporados en los Sistemas Operativos y a veces en el propio almacén de los navegadores (ej. Firefox).
- Tipos de Certificados SSL/TLS (Seguridad vs. Validación): 42

|                                    | ¿Cifrado? | ¿Candado? | ¿Validación de dominio? | ¿Validación de dirección postal? | ¿Validación de identidad? | ¿Barra de navegación verde? |
|------------------------------------|-----------|-----------|-------------------------|----------------------------------|---------------------------|-----------------------------|
| <b>DV</b><br>Dominio validado      | SI        | SI        | SI                      | NO                               | NO                        | NO                          |
| <b>OV</b><br>Organización validada | SI        | SI        | SI                      | SI                               | Buena                     | NO                          |
| <b>EV</b><br>Validación Extendida  | SI        | SI        | SI                      | SI                               | Muy Buena                 | SI                          |

Otros Tipos: Multi-dominio (SANs - Nombres Alternativos de Asunto) y Wildcard SSL (.sudominio.com).

#### Certificados de Cliente

- Definición: Documento electrónico X.509 emitido por una CA para autenticar la identidad de un cliente.
- Uso: Identifica digitalmente a un cliente/persona y se importa en el almacén de certificados. Es un factor de seguridad de tipo "algo que tiene".
- Validación: El servidor verifica el certificado del cliente contra una CA alojada en el mismo.
- Validación Fuerte: Se puede combinar con usuario y contraseña (factor "algo que sabe") para una validación fuerte.

## 10.5 HTTPS HANDSHAKE

El Handshake (protocolo dentro de la suite SSL/TLS) utiliza algoritmos de clave asimétrica para generar e intercambiar información que permita obtener una clave simétrica para cifrar el tráfico.

Pasos del Handshake (Proceso Simplificado):

1. Negociación
2. Intercambio de Certificado
3. Generación de Claves
4. Conexión Cifrada

## 10.6 LET'S ENCRYPT

- Definición: Una Autoridad de Certificación (CA) gratuita, automatizada y abierta ofrecida por el ISRG (Internet Security Research Group) sin fines de lucro.
- Importancia: Ha sido un actor importante desde su disponibilidad en diciembre de 2015, contribuyendo significativamente a la expansión de sitios web cifrados.
- Tipos de Certificados: Solo ofrece certificados de tipo DV (Dominio Validado) y SAN (Multi-dominio). No planean emitir certificados OV (Organización Validada) o EV (Validación Extendida).

- Estadísticas (Tendencias):
  - Cifrado General: Los sitios web tienden a estar 100% cifrados.
  - Uso de Let's Encrypt: El 60% de los sitios web monitoreados utiliza Let's Encrypt, lo que representa una cuota de mercado del 63.5%.

## **10.7 CONCLUSIONES Y TENDENCIAS**

- La tendencia es que los sitios web estén 100% cifrados.
- Los navegadores penalizan los sitios no seguros (HTTP).
- Los buscadores (como Google desde 2014) posicionan negativamente a HTTP.
- Hay un creciente *Phishing* con certificados http.
- El Certificado SSL de Validación Extendida (EV) tiende a tomar más relevancia.
- La versión más reciente y segura es TLS 1.3 (RFC 8446, 2018).